# Emergency Training Simulator POC

Submission Packet

## Emergency Training Simulator POC Report

**Submission report**

Source: docs/reports/project-overview-report.md

Prepared for review, discussion, and project handoff.

Markdown stays editable. This PDF is the clean reading copy.

Boundary: this is still a POC, not operational or clinical guidance.

# Emergency Training Simulator POC Report

Prepared as a project progress report for Dr. Melissa Cameron.

## Table of contents

1. Executive summary
2. How the project started
3. Full project north star
4. Why the first POC is smaller
5. Architecture at a glance
6. Current POC scope
7. Current implementation
8. Review and replay as the proof point
9. Validation boundary
10. Security and trust boundary
11. Relationship to future Unreal work
12. Relationship to future assets and physics
13. Solo-developer checkpoint plan
14. Rough timeline
15. Two-track roadmap
16. Recommended next steps
17. Closing

## Executive summary

The long-term project vision is an AI-driven, real-time adaptive emergency-response training platform for firefighters and EMTs. The full system points toward agentic AI scenario generation, immersive training rooms, physical mannequins, multimodal sensing, structured verification and validation, scalable hub-and-spoke deployment, and post-scenario review.

This repository is not the full platform yet. It is the first proof-of-concept foundation underneath that larger direction.

The current POC proves a smaller but important loop: a manual or scripted trainee action reaches the backend, changes deterministic scenario state, gets checked by a validation layer, returns client-ready state, writes a JSONL log, and can be replayed and reviewed afterward.

The project is being built by one solo developer right now. That matters. The full project vision is large enough to include multiple research and engineering tracks, so this repo deliberately turns the work into smaller checkpoints that can be built, tested, shown, and reviewed.

## How the project started

The starting point was a broad project overview for an AI-driven adaptive emergency-response training simulator.

That original direction included:

- AI-assisted and agentic scenario generation
- real-time scenario adaptation
- immersive training spaces

- Unreal or similar real-time visualization
- physical mannequins or props
- sensing of trainee behavior
- validation against structured references
- post-scenario review
- scalable deployment across sites

That is the long-term direction. It is not what a first solo-developer POC should try to fully implement.

The first practical decision was to separate the full vision from the first proof.

The full vision asks:

**What could the complete adaptive training platform eventually become?**

The first POC asks:

**Can the software keep track of a scenario, validate toy actions, log the session, and explain what happened afte**

## Full project north star

The full project eventually aims to support a closed-loop training experience:

```
trainee speaks, moves, and performs procedures
-> sensing captures behavior
-> AI interprets actions
-> scenario state adapts
-> validation checks safety and coherence
-> immersive environment updates
-> session data is logged
-> review supports learning and system improvement
```

Future capabilities include AI and agentic scenario adaptation, multimodal sensing, projection mapping, mannequin integration, cloud services, distributed training sites, and validation against structured clinical or training references.

Those are future full-system goals. They are not current repo capabilities.

## Why the first POC is smaller

The first POC is intentionally narrow because the full system depends on a reliable truth layer.

Before adding realistic visuals, generated assets, AI calls, sensors, or physics, the project needs to answer simpler questions:

- What action was attempted?
- Was the action accepted, warned, or rejected?
- What scenario state changed?
- What environmental cues mattered?
- What was logged?
- Can the ordered session be replayed?
- Can an instructor understand the session afterward?

If those answers are unclear, later immersive work can look impressive while hiding a weak foundation.

The current repo therefore focuses on the backend loop first. That is a scope-control choice, not a reduction of the long-term ambition.

## Architecture at a glance

The full project vision and the current POC can be understood as two connected layers:

```
Full-system direction
AI scenario generation
+ immersive rooms
+ multimodal sensing
+ validation framework
+ hub-and-spoke deployment
|
v
Current POC foundation
manual action
-> backend event
-> deterministic state update
-> POC validation
-> client state message
-> JSONL log
-> replay and review
```

The current repo builds the lower layer first so that future AI, sensing, Unreal, and deployment work have a trusted state and review foundation.

## Current POC scope

The current repo focuses on one trusted backend loop:

```
manual/stubbed trainee action
-> FastAPI backend receives event
-> deterministic scenario engine updates state
-> POC validation checks rule conformance
-> WebSocket client receives state
-> JSONL log is written
-> replay and review explain what happened
```

This scope is intentionally narrow. The goal is to make the core logic testable, repeatable, and explainable before adding AI, Unreal, sensing, projection, or real physics.

## Current implementation

The current implementation includes:

- FastAPI health endpoint
- WebSocket session endpoint
- deterministic YAML scenario engine
- Pydantic message models
- JSON schemas and examples
- POC validation rules
- local JSONL session logs
- replay script
- improved post-scenario review summary
- one-command local demo
- future Unreal client contract
- text-only Unreal display preview
- JSON-to-debug-text bridge for the first Unreal panel
- environment object contract

- basic security guardrails
- lightweight GitHub Actions CI

The repo now has two small deterministic POC scenarios. The smoke-room scenario includes a wooden chair and an entry door as environment cues. The chair can block the path. The door can be open or closed. The happy path moves the chair, opens the door, enters the room, and removes the occupant. The warning/error path shows what happens when the trainee ignores sequence or environmental cues.

The blocked-exit hallway scenario adds a second toy environment with a hallway obstacle and exit-door state. This helps prove the scenario engine is not hardcoded to one room.

This scenario is a software fixture only. It is not emergency guidance.

## Review and replay as the proof point

The strongest result so far is not just that the demo runs.

The strongest result is that the system can explain what happened.

The review summary now includes:

- session basics
- final phase and outcome
- end condition status
- validation counts
- replay verification
- action timeline
- environment cues
- warnings
- errors
- key observations
- reviewer takeaways

For example, the warning/error path can show that a trainee attempted actions too early, that the entry door remained closed, that warnings/errors were recorded, and that replay verified the ordered events.

This matters because training systems need debriefing. A visually impressive simulation is less useful if it cannot explain the session afterward.

## Validation boundary

Current validation means POC rule conformance only.

It answers:

**Did this action follow this toy scenario’s deterministic rules?**

It does not claim:

- clinical validity
- medical correctness
- official EMT protocol authority
- firefighter protocol authority
- certification scoring
- real-world operational guidance

The full project may eventually include a structured verification and validation framework trained or checked against validated references. That work belongs to a later stage.

## Security and trust boundary

The backend is the source of truth.

A client can request an action, but the backend decides:

- current scenario state
- validation status
- warnings and errors
- final outcome
- replay/review output

The current repo includes basic POC guardrails:

- safe identifier checks
- safe scenario/action names
- small payload limits
- sequence checks
- duplicate event rejection
- max active sessions
- max actions per session
- clearer WebSocket error codes

This is not production security. There is no authentication, authorization, TLS, cloud hardening, real rate limiting, or real data policy yet.

## Relationship to future Unreal work

Unreal is treated as a future client, not the source of truth.

The future Unreal client should display backend-owned state and send trainee actions. It should not decide validation results, final outcomes, or review summaries.

The current repo includes a text-only future Unreal contract, display preview, checked-in sample display JSON, and a JSON-to-debug-text bridge. The bridge lets Unreal show a generated text panel before it parses JSON directly.

The current Unreal checkpoint is intentionally small: the hardcoded debug panel can show the same text, and the next step is making Unreal read the generated text file automatically.

The repo does not build a full Unreal scene.

## Relationship to future assets and physics

The current chair and door are scenario objects and metadata, not generated 3D assets.

Future generated assets may come from tools like Hunyuan 3D or similar pipelines, but assets should be reviewed manually before use. Blender is likely useful later as the review and cleanup tool before Unreal import.

Physics is also future work. The current repo can represent physics preview metadata, but it does not simulate gravity, collision, tipping, mass, or force.

## Solo-developer checkpoint plan

The project is being staged as a solo-developer effort.

That means each slice should:

- prove one clear idea
- stay testable locally
- avoid adding expensive tools before they have a job
- preserve the backend as the source of truth
- produce something Dr. Cameron can review
- leave the next slice easier than the previous one

The current checkpoint sequence is:

1. repo scaffold and project boundaries
2. deterministic backend loop
3. replay and review proof
4. richer deterministic environmental cues
5. second scenario to prove reuse
6. Unreal text display path
7. Unreal file/JSON display
8. live Unreal backend display
9. simple scene shell
10. reviewed asset workflow
11. later physics, AI, sensing, validation, and deployment experiments

This sequence keeps the project moving toward the full vision without pretending the first POC is the full simulator.

## Rough timeline

This timeline is a planning guide, not a promise.

Actual pacing depends on solo-developer time, Unreal setup friction, compute access, research direction from Dr. Cameron, and whether the next milestones stay small.

### Current checkpoint: backend proof and review loop

Status: mostly working locally.

The repo can already run deterministic backend scenarios, produce validation results, write JSONL logs, replay sessions, and generate review summaries.

### Near term: Unreal display path

Target: make the backend state visible without building a full scene.

Planned steps:

1. make Unreal read hallway_debug_panel.sample.txt
2. make Unreal parse hallway_display_state.sample.json
3. connect Unreal to the backend WebSocket
4. keep all scenario authority in the backend

### Mid term: simple visual shell and reviewed assets

Target: make the demo easier to understand visually while staying controlled.

Possible steps:

- simple Unreal room or hallway shell
- visible debug/status panel
- basic object placeholders
- Blender/manual asset review workflow
- first reviewed chair/door/obstacle assets

### Later research phases

Target: explore the larger original vision only after the foundation is stable.

Possible later phases:

- generated assets such as Hunyuan 3D candidates
- real physics experiments
- trainee sensing experiments
- AI-assisted adaptation
- stronger validation framework
- cloud and hub-and-spoke planning

## Two-track roadmap

The project now has two tracks.

POC track:

- keep the deterministic backend loop stable
- improve replay and review
- refine message contracts
- add small deterministic scenarios
- strengthen UX and instructor review flow
- prepare for a thin Unreal display client

Full-system track:

- AI scenario generation and adaptation
- validation framework expansion
- multimodal sensing
- immersive rooms and projection mapping
- mannequin integration
- real physics and environment simulation
- hub-and-spoke deployment
- cloud services and production security

The POC track should continue to move first. It creates the foundation that the full-system track will need.

## Recommended next steps

The next useful work should stay close to the proof point:

1. improve instructor/reviewer examples
2. add a small evaluation rubric for Dr. Cameron
3. make Unreal read the generated debug text file automatically
4. parse the display JSON directly in Unreal after the text path works
5. keep the backend as source of truth

Unreal, generated assets, AI calls, sensing, and deployment should wait until the contracts and review loop are stable.

## Closing

This repo is small on purpose.

It does not try to be the final adaptive training platform yet. It proves the first trustworthy backend loop: action in, state update, validation result, client message, log,

replay, and review.

That foundation is what makes the larger project believable.